Problem 1(a) – Set Cover (Greedy Selection)

Let each word be treated as a set of letters:

$$\begin{aligned}
\text{dog} &= \{d, o, g\},\\
\text{goat} &= \{g, o, a, t\},\\
\text{ate} &= \{a, t, e\},\\
\text{dot} &= \{d, o, t\},\\
\text{timer} &= \{t, i, m, e, r\},\\
\text{road} &= \{r, o, a, d\},\\
\text{tread} &= \{t, r, e, a, d\},\\
\text{eye} &= \{e, y\}.
\end{aligned}$$

The universe of letters is

$$U = \{a, d, e, g, i, m, o, r, t, y\}.$$

Using the greedy heuristic (pick the set covering the largest number of uncovered elements; break ties by first appearance):

1. Choose **timer**: covers $\{t, i, m, e, r\}$ (5 uncovered).
2. Choose **dog**: covers $\{d, o, g\}$ (3 new uncovered).
3. Choose **goat**: covers $\{a\}$ (1 new uncovered).
4. Choose **eye**: covers $\{y\}$ (1 new uncovered).

All 10 letters are now covered.

Final greedy order: timer, dog, goat, eye. **Newly covered elements:** 5, 3, 1, 1 respectively.

Problem 2(a) – Set Cover and Vertex Cover Equivalence

Given an undirected graph $G = (V, E)$, define the universe $U = E$ and for each vertex $v \in V$, define

$$S_v = \{e \in E \mid e \text{ is incident on } v\}.$$

Claim. A subset $C \subseteq V$ is a vertex cover of G if and only if the collection $\{S_v \mid v \in C\}$ is a set cover of U.

Proof.

- ($\Rightarrow$) If C is a vertex cover, then every edge $e \in E$ is incident to some vertex in C. Hence for every $e \in U$, there exists $v \in C$ such that $e \in S_v$. Therefore, $\{S_v \mid v \in C\}$ covers U.
- ($\Leftarrow$) Conversely, if $\{S_v \mid v \in C\}$ covers U, then for every edge $e \in E$, there is a vertex $v \in C$ such that $e \in S_v$. Thus v is an endpoint of e, and every edge has at least one endpoint in C. Hence C is a vertex cover.

Problem 2(b) – Minimum Sizes Correspondence

Each vertex $v \in V$ corresponds directly to the subset $S_v \subseteq U$. Thus, any vertex cover C corresponds to the set cover $\{S_v : v \in C\}$ with identical size $|C|$.

Conversely, for any set cover $\mathcal{S}$, we can define

$$C = \{v \mid S_v \in \mathcal{S}\}$$

with $|C| = |\mathcal{S}|$. Therefore:

$$\text{Minimum Vertex Cover size in } G = \text{Minimum Set Cover size in this instance.}$$

A minimum solution in one problem maps directly to a minimum solution in the other.

Problem 3(a) – Huffman Encoding: {0, 10, 11}

This prefix code is valid (no codeword is a prefix of another). Huffman can produce this encoding by merging the two least frequent symbols first.

Example frequencies:

$$f_a = 100, \quad f_b = 1, \quad f_c = 1.$$

Huffman merges b and c first, producing codes:

$$a : 0, \quad b : 10, \quad c : 11.$$

Hence $\{0, 10, 11\}$ is possible.

Problem 3(b) – Huffman Encoding: {0, 00, 10}

The code $\{0, 00, 10\}$ is **not prefix-free**, since "0" is a prefix of "00". Huffman always produces prefix-free codes, so this encoding is **impossible**.

Problem 3(c) – Huffman Encoding: {00, 10, 11}

Although this set is prefix-free, it cannot arise from Huffman's algorithm for 3 symbols.

For 3 symbols, the Huffman tree must have leaf depths $\{1, 2, 2\}$ (one leaf at depth 1, two at depth 2). A binary tree with all three leaves at depth 2 would require 4 leaves. Hence, $\{00, 10, 11\}$ cannot come from Huffman encoding.

Problem 4(a) – Longest Huffman Codeword Length

For n symbols with frequencies $f_1, f_2, \ldots, f_n$, the longest codeword in Huffman encoding can have length at most $n - 1$.

Construction achieving the bound: Use a degenerate chain of merges by assigning frequencies following a rapidly increasing pattern, such as Fibonacci numbers:

$$f_i = F_i, \quad F_1 = 1,\ F_2 = 1,\ F_3 = 2, \ldots$$

Huffman merges the smallest two symbols repeatedly, producing a tree shaped as a single path. Thus, one codeword attains length $n - 1$.

Upper bound argument: A full binary tree with n leaves has height at most $n - 1$, since each merge introduces one internal node. Therefore, the maximum Huffman codeword length is $\boxed{n - 1}$.

Problem 4(b) – Frequency Bound for Length-1 Codeword

Claim. If every symbol has frequency less than $\frac{1}{3}$, then no symbol can have a length-1 codeword.

Proof. Suppose a symbol s has a length-1 codeword. Then in the final Huffman tree, the root has:

Left child: symbol s with frequency p_s, Right child: remaining symbols with total $1 - p_s$.

Since the right child contains at least two symbols, one of them has frequency $\geq (1 - p_s)/2$. For s to remain unmerged until the last step, we must have:

$$p_s \geq \frac{1 - p_s}{2} \quad \Rightarrow \quad p_s \geq \frac{1}{3}.$$

Hence, if $p_s < \frac{1}{3}$ for all s, no codeword of length 1 can exist.

Problem 5 – Huffman Tree Height

Claim. For $n \geq 2$, the height (maximum codeword length) of a Huffman tree is at most $n - 1$.

Proof by induction on n:

Base case ($n = 2$): The tree has one merge and height $1 = n - 1$. Holds true.

Inductive step: Assume the claim holds for $n - 1$ symbols. When Huffman merges the two smallest frequencies, they form one new node. The new instance has $(n - 1)$ nodes, whose tree height by the inductive hypothesis is at most $(n-2)$. Adding one merge increases the height by at most 1, giving:

$$h_n \leq (n - 2) + 1 = n - 1.$$

Hence, by induction, the height of the Huffman tree is at most $n - 1$.